



PYTHON · DAQS · DATA EDUCATION

Introduction to Python

Programming

A comprehensive, hands-on curriculum for aspiring Python developers.

Duration

10 Modules · ~3 Months

Level

Beginner to Intermediate

Format

Self-paced · Project-based

Experience

None Required

```
import numpy as np          # numerical computing
import pandas as pd         # data manipulation
def analyse(df):            # define a function
    return df.describe()    # summary statistics
for i in range(10):         # iteration
    print(f'Value: {i}')    # formatted output
```

Overview

Python is the world's most popular programming language — powering data science, artificial intelligence, web development, automation, and financial engineering. The **DAQS Introduction to Python** curriculum provides a structured, project-driven pathway from absolute beginner to confident practitioner, equipping students with skills immediately applicable in professional and research environments.

Write Clean Code Master Python syntax, style, and PEP 8 from day one.	Work with Data Use NumPy, Pandas, and Matplotlib on real datasets.	Build Applications Develop OOP programs, scripts, and CLI tools.	Deploy to GitHub Version-control and publish a professional portfolio.
10 Modules	3–4 Months	10+ Capstones	0 Prerequisites

Who Is This For?

Absolute beginners — No prior coding experience — we start from scratch.

Career changers — Break into data science, AI, or software engineering.

Analysts & researchers — Automate tasks and unlock data insights with Python.

DAQS students — Foundation course for advanced DAQS AI/ML programmes.

Syllabus Outline

What You'll Learn

LEVEL 1 — Python Fundamentals

Modules 01–04

01

Introduction to Programming & Python Basics

- Set up Python, VS Code, and virtual environments from scratch.
- Learn core data types: int, float, str, bool, NoneType.
- Use variables, assignment, and PEP 8 naming conventions.
- Perform arithmetic, string concatenation, and type conversion.
- Communicate with users via `print()` and `input()`.
- Write comments and understand why clean, readable code matters.
- Distinguish scripts from interactive shells (REPL).

Capstone Project

Investment Calculator

Build a financial calculator using variables, data types, and formatted output.

02

Control Flow — Decisions and Loops

- Write if, elif, and else statements to guide program logic.
- Use comparison (`==`, `!=`, `<`, `>`) and logical (and, or, not) operators.
- Understand truthy and falsy values in Python.
- Build for loops to iterate over ranges and sequences.
- Build while loops with correct termination conditions.
- Control loops with `break`, `continue`, and `pass`.
- Apply nested loops and trace execution with print debugging.

Capstone Project

Number Guessing Game

A guessing game using loops and conditionals, tracking attempts with hints.



03

Data Structures — Lists, Tuples, Sets & Dictionaries

- Create and manipulate lists: slicing, appending, sorting, removing.
- Understand mutability (list) vs immutability (tuple).
- Use sets for deduplication and O(1) membership testing.
- Build and query dictionaries: add, update, delete, iterate.
- Write list and dictionary comprehensions for concise transforms.
- Nest data structures: lists of dicts, dicts of lists.
- Use the collections module: Counter, defaultdict, OrderedDict.

Capstone Project

Student Records System

Dictionary-based grade manager with search, sort, and summary using comprehensions.

04

Functions, Scope & Modular Code

- Define functions: parameters, defaults, and return statements.
- Use *args and **kwargs for flexible function signatures.
- Master the LEGB scope rule: local, enclosing, global, built-in.
- Write pure functions and understand side effects.
- Apply recursion: factorial, Fibonacci, and tree traversal.
- Use lambda functions and the Python Standard Library (math, datetime, os).
- Organise code into .py modules and write docstrings.

Capstone Project

Utilities Library

A reusable module of functions (stats, formatters, date tools) imported across scripts.

LEVEL 2 — Applied Python

Modules 05–07

05

String Manipulation & Regular Expressions

- Use string methods: split(), join(), strip(), replace(), find().
- Format strings with f-strings, .format(), and %-formatting.
- Understand Unicode, encoding, and string immutability.
- Validate and parse emails, phone numbers, and dates.
- Introduce the re module: search(), match(), findall(), sub().
- Write regex patterns: character classes, quantifiers, groups, anchors.
- Apply regex to real-world data cleaning tasks.

Capstone Project

Data Cleaning Script

Reads messy raw text, validates with regex, standardises, and outputs a clean result.

06

File Handling & Exception Management

- Open, read, write, and append files with context managers (with).
- Work with CSV files using the csv module.
- Parse and generate JSON data with the json module.
- Navigate the filesystem using os and pathlib.
- Understand the exception hierarchy and write try/except/finally blocks.
- Raise custom exceptions with informative error messages.
- Apply logging as a professional alternative to print debugging.

Capstone Project

CSV Report Generator

Reads a raw CSV, handles malformed rows, and writes a JSON summary report.

07

Object-Oriented Programming (OOP)

- Understand OOP pillars: Encapsulation, Inheritance, Polymorphism, Abstraction.
- Define classes with __init__, instance variables, and methods.
- Distinguish instance, class (@classmethod), and static (@staticmethod) methods.
- Implement inheritance — extend and override parent class methods.
- Use super() to call parent constructors cleanly.
- Implement dunder methods: __str__, __repr__, __len__, __eq__.
- Model real-world entities with UML-style class thinking.

Capstone Project

Banking System

Account, SavingsAccount, LoanAccount classes using inheritance and polymorphic methods.

LEVEL 3 — Data & Deployment

Modules 08–10



08

Working with Data — NumPy & Pandas

- Understand why NumPy arrays outperform Python lists numerically.
- Create, index, slice, reshape, and broadcast NumPy arrays.
- Load DataFrames from CSV, JSON, and dictionaries.
- Inspect data: `head()`, `info()`, `describe()`, `shape`, `dtypes`.
- Filter and transform with `loc[]`, `iloc[]`, and boolean masks.
- Handle missing values: `isnull()`, `dropna()`, `fillna()`.
- Aggregate with `groupby()`, `agg()`, `pivot_table()`, `merge()`, `concat()`.

Capstone Project

Exploratory Data Analysis

Full EDA on a real dataset: load, clean, explore, group, aggregate, and summarise.

09

Data Visualisation — Matplotlib & Seaborn

- Understand the Matplotlib figure/axes architecture and plot anatomy.
- Create line charts, bar charts, histograms, scatter plots, and pie charts.
- Customise plots: titles, labels, legends, colours, gridlines.
- Build subplots with `plt.subplots()` for multi-panel dashboards.
- Use Seaborn: boxplot, violin, heatmap, pairplot for statistical visuals.
- Detect and visualise outliers using Z-score and IQR colour-coding.
- Save figures as PNG/PDF at publication quality with `savefig()`.

Capstone Project

Analytical Dashboard

Multi-chart dashboard combining histograms, boxplots, scatter, and a heatmap.

10

Capstone — End-to-End Python Project

- Apply all modules in one cohesive project: ingest, clean, analyse, report.
- Build a CLI application with argument parsing using `argparse`.
- Structure a project: `src/`, `tests/`, `README.md`, `requirements.txt`.
- Write unit tests with `unittest` or `pytest`.
- Use Git for version control: `commit`, `branch`, `merge`, push to GitHub.
- Document with a README, inline docstrings, and a usage guide.
- Package dependencies with `pip` and `requirements.txt`.

Capstone Project

Portfolio Project

A complete, tested, documented, version-controlled Python project on GitHub.

Potential Career Paths

After completing this programme, graduates are well-positioned for the following roles — directly or after advancing to DAQS higher-level courses:

Data Analyst

Clean, analyse, and report on business data using Python and Pandas.

Junior Data Scientist

Apply ML libraries to predictive problems — built on these foundations.

Python Developer

Build scripts, automation tools, and backend services.

ML Engineer (Junior)

Work with AI pipelines using Python as the language of AI.

Automation Engineer

Automate business processes with scripting and API integration.

Research Programmer

Support research with data processing and reproducible analysis.

Technical Requirements

- › **Computer:** Any Windows, macOS, or Linux machine made after 2015.
- › **RAM:** 4 GB minimum (8 GB recommended for data work).
- › **Software:** Python 3.10+, VS Code, Git — all free to install.
- › **Internet:** Required for materials, GitHub, and package installation.
- › **Cost:** No additional licences required — all tools are open source.

Take the Next Step



Whether starting fresh or formalising existing skills, the DAQS Introduction to Python gives you the foundation, confidence, and portfolio to move forward.

Contact Us

Discuss your goals and find the right path.
ncube.t@daqstech.com

Visit Our Website

Explore all DAQS programmes and resources.
www.daqstech.com

Enrol Now

Ready to start? Contact us to begin enrolment.
ncube.t@daqstech.com